```
import duckdb
con = duckdb.connect()
```

```
import gdown

file_id = "1RxeikLQHjYEJCewMtxpbNVlQ1FcQcC9s"
url = f"https://drive.google.com/uc?id={file_id}"

output = "/content/farmer_questions.csv"

gdown.download(url, output, quiet=False)

print("Download complete:", output)
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1RxeikLQHjYEJCewMtxpbNVlQ1FcQcC9s
From (redirected): https://drive.google.com/uc?id=1RxeikLQHjYEJCewMtxpbNVlQ1FcQcC9s&confirm=t&uuid=35a8e995-9c5d-4aee-b92f-fad214589012
To: /content/farmer_questions.csv
100%|██████████| 7.25G/7.25G [01:26<00:00, 83.9MB/s]Download complete: /content/farmer_questions.csv
```

```
import duckdb

# connect to the DuckDB file
con = duckdb.connect('/content/producers_direct.duckdb')
```

```
import duckdb
import pandas as pd

# Connect to in-memory DuckDB
con = duckdb.connect()

# Load CSV into DuckDB table (on-the-fly)
con.execute("""
    CREATE OR REPLACE TABLE farmer_questions AS
    SELECT * FROM read_csv_auto('/content/farmer_questions.csv')
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

```
con.execute("""
SELECT *
```

```
SELECT *
FROM farmer_questions
WHERE
    question_id IS NOT NULL
    AND question_user_id IS NOT NULL
    AND question_language IS NOT NULL
    AND question_content IS NOT NULL
    AND question_topic IS NOT NULL
    AND question_sent IS NOT NULL
    AND response_id IS NOT NULL
    AND response_user_id IS NOT NULL
    AND response_language IS NOT NULL
    AND response_content IS NOT NULL
    AND response_topic IS NOT NULL
    AND response_sent IS NOT NULL
    AND question_user_type IS NOT NULL
    AND question_user_status IS NOT NULL
    AND question_user_country_code IS NOT NULL
    AND question_user_gender IS NOT NULL
    AND question_user_dob IS NOT NULL
    AND question_user_created_at IS NOT NULL
    AND response_user_type IS NOT NULL
    AND response_user_status IS NOT NULL
    AND response_user_country_code IS NOT NULL
    AND response_user_gender IS NOT NULL
    AND response_user_dob IS NOT NULL
    AND response_user_created_at IS NOT NULL
LIMIT 10
""").df()
```

	question_id	question_user_id	question_language	question_content	question_topic	question_sent	response_id	response_user_id	response_lan
0	3849862	93494	eng	q#wts price of potato n kisumu	potato	2017-11-22 13:26:39+00:00	3851648	98760	
1	3850558	102473	eng	Q isit better better,to torrorate the pigs if ...	pig	2017-11-22 14:23:42+00:00	3858755	117705	
2	3853087	135417	eng	Q#Is it the right season to caltivate capsicum...	capsicum	2017-11-22 16:26:09+00:00	3855484	56273	
3	3859695	108971	eng	Q..a bird known as a pentum..how does it look ...	bird	2017-11-23 02:05:35+00:00	3863907	152917	
4	3859695	108971	eng	Q..a bird known as a pentum..how does it look ...	pigeon	2017-11-23 02:05:35+00:00	3863907	152917	
5	3861230	146723	eng	Q which materials are needed to start poultry	poultry	2017-11-23 04:44:18+00:00	3937688	232124	

```
con.execute("""
    SELECT DISTINCT
        EXTRACT(YEAR FROM question_user_created_at) AS year
    FROM farmer_questions
    WHERE question_user_created_at IS NOT NULL
    ORDER BY year
""").df()
```

8	3872184	135053	eng	Q are there any farmer dealing in rabit keepin...	rabbit	2017-11-23 16:08:40+00:00	3872218	95624	
9	3872304	37044	eng	Q,how Can I Structure A House For Ten Pigs?	pig	2017-11-23 16:15:11+00:00	3872400	44247	

10 rows × 24 columns

	year
2	2016
3	2017
4	2018
5	2019
6	2020
7	2021
8	2022

```
con.execute("DESCRIBE farmer_questions").df()
```

```
con.execute("""
SELECT
  question_id,
  question_language,
  question_user_country_code,
  question_topic,
  question_sent,
  LEFT(question_content, 140) AS question_preview,
  response_language,
  LEFT(response_content, 140) AS response_preview
FROM farmer_questions
LIMIT 20;
""").df()
```

8	response_language	VARCHAR	YES	None	None	None
9	response_content	VARCHAR	YES	None	None	None
10	response_topic	VARCHAR	YES	None	None	None
11	response_sent	TIMESTAMP WITH TIME ZONE	YES	None	None	None
12	question_user_type	VARCHAR	YES	None	None	None
13	question_user_status	VARCHAR	YES	None	None	None
14	question_user_country_code	VARCHAR	YES	None	None	None
15	question_user_gender	VARCHAR	YES	None	None	None
16	question_user_dob	DATE	YES	None	None	None
17	question_user_created_at	TIMESTAMP WITH TIME ZONE	YES	None	None	None
18	response_user_type	VARCHAR	YES	None	None	None
19	response_user_status	VARCHAR	YES	None	None	None
20	response_user_country_code	VARCHAR	YES	None	None	None
21	response_user_gender	VARCHAR	YES	None	None	None
22	response_user_dob	DATE	YES	None	None	None
23	response_user_created_at	TIMESTAMP WITH TIME ZONE	YES	None	None	None

	question_id	question_language	question_user_country_code	question_topic	question_sent	question_preview	response_language	
0	3849056	nyn	ug	None	2017-11-22 12:25:03+00:00	E ABA WEFARM OFFICES ZABO NIZISHANGWA NKAHI?	nyn	E!2
1	3849061	eng	ug	None	2017-11-22 12:25:05+00:00	Q this goes to wefarm. is it possible to get f...	eng	Q1 wh
2	3849077	nyn	ug	cattle	2017-11-22 12:25:08+00:00	E ENTE YANJE EZAIRE ENYENA YASHOBERA. \nOBWIRE...	nyn	Muha masha
3	3849077	nyn	ug	cattle	2017-11-22 12:25:08+00:00	E ENTE YANJE EZAIRE ENYENA YASHOBERA. \nOBWIRE...	nyn	Muha masha
4	3849077	nyn	ug	cat	2017-11-22 12:25:08+00:00	E ENTE YANJE EZAIRE ENYENA YASHOBERA. \nOBWIRE...	nyn	Muha masha
5	3849077	nyn	ug	cat	2017-11-22 12:25:08+00:00	E ENTE YANJE EZAIRE ENYENA YASHOBERA. \nOBWIRE...	nyn	Muha masha
6	3849078	nyn	ug	None	2017-11-22 12:25:09+00:00	E. Radio ezimwagaba nituzituga tunta ariho aba...	nyn	Mukoze
7	3849082	swa	ke	poultry	2017-11-22 12:25:10+00:00	S dawa ya viroboto.kwa kuku	swa	S Tulikua
8	3849082	swa	ke	poultry	2017-11-22 12:25:10+00:00	S dawa ya viroboto.kwa kuku	eng	
9	3849084	eng	ke	rabbit	2017-11-22 12:25:10+00:00	Q-i have stock rabbit's urine for 5 weeks mash...	eng	
					2017-11-22	S		

```
con.execute("""
SELECT
  COUNT(*) AS n_rows,
  COUNT(DISTINCT question_id) AS n_unique_questions,
  ROUND(100.0 * COUNT(DISTINCT question_id) / COUNT(*), 2) AS pct_rows_that_are_unique_questions
FROM farmer_questions;
""").df()
```

	n_rows	n_unique_questions	pct_rows_that_are_unique_questions
0	20304843	5865819	28.89

Only 28.89% of rows represent unique questions, confirming that analysis must be conducted at the question level to avoid inflated counts from multiple responses per question.

```
con.execute("""
SELECT
  COUNT(*) AS n_questions_with_multiple_responses
FROM (
  SELECT question_id
  FROM farmer_questions
  WHERE question_id IS NOT NULL AND response_id IS NOT NULL
  GROUP BY 1
  HAVING COUNT(DISTINCT response_id) > 1
) t;
""").df()
```

	n_questions_with_multiple_responses
0	4241532

Out of approximately 5.87 million unique questions, 4.24 million received more than one response, confirming that response-level comparisons are meaningful and that row-level counts would inflate question totals.

```
con.execute("""
SELECT COUNT(*) AS n_duplicate_pairs
FROM (
  SELECT question_id, response_id
  FROM farmer_questions
  WHERE question_id IS NOT NULL AND response_id IS NOT NULL
  GROUP BY 1,2
  HAVING COUNT(*) > 1
) t;
""").df()
```

	n_duplicate_pairs
0	2709627

There are about 2.7 million cases where the same answer to the same question was stored multiple times as separate rows.

```
con.execute("""
SELECT
  COUNT(*) AS n_rows,
  SUM(CASE WHEN question_topic IS NULL OR TRIM(question_topic) = '' THEN 1 ELSE 0 END) AS n_topic_missing,
  ROUND(100.0 * SUM(CASE WHEN question_topic IS NULL OR TRIM(question_topic) = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_topic_missing
FROM farmer_questions;
""").df()
```

	n_rows	n_topic_missing	pct_topic_missing
0	20304843	3537729.0	17.42

Approximately 17.4% of rows lack a question topic, so questions with missing topics will be either excluded from topic-based analyses or grouped into an “Unknown” category.

```
con.execute("""
SELECT question_language, COUNT(*) AS n_rows
FROM farmer_questions
GROUP BY 1
ORDER BY n_rows DESC;
""").df()
```

	question_language	n_rows
0	eng	11976781
1	swa	6480848
2	nyn	1157228
3	lug	689986

```
q_text_missing = con.execute("""
SELECT
  COUNT(*) AS n_rows,
  SUM(CASE WHEN question_content IS NULL THEN 1 ELSE 0 END) AS n_null_question,
  SUM(CASE WHEN question_content IS NOT NULL AND TRIM(question_content) = '' THEN 1 ELSE 0 END) AS n_blank_question,
```

```
  ROUND(100.0 * SUM(CASE WHEN question_content IS NULL OR TRIM(question_content) = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_missing_or_blank
FROM farmer_questions;
""").df()

q_text_missing
```

	n_rows	n_null_question	n_blank_question	pct_missing_or_blank
0	20304843	0.0	0.0	0.0

```
q_text_noise = con.execute("""
SELECT
  COUNT(*) AS n_nonnull_question_rows,
  SUM(CASE WHEN question_content LIKE '%\n%' THEN 1 ELSE 0 END) AS n_has_newline,
  SUM(CASE WHEN question_content LIKE '%  %' THEN 1 ELSE 0 END) AS n_has_double_space
FROM farmer_questions
WHERE question_content IS NOT NULL;
""").df()

q_text_noise
```

	n_nonnull_question_rows	n_has_newline	n_has_double_space
0	20304843	349103.0	2488669.0

All questions contain non-empty text, so no filtering was required for missing question content.

```
resp_lang = con.execute("""
SELECT response_language, COUNT(*) AS n_rows
FROM farmer_questions
GROUP BY 1
ORDER BY n_rows DESC;
""").df()

resp_lang
```

	response_language	n_rows
0	eng	11939866
1	swa	6589074
2	nyn	1094640
3	lug	681219
4	spa	44

Next steps: Generate code with resp_lang | New interactive sheet

```
con.execute("SHOW TABLES").df()
```

	name
0	farmer_questions

```
con.execute("""
CREATE OR REPLACE VIEW question_level AS
SELECT
  question_id,
  ANY_VALUE(question_language) AS question_language,
  ANY_VALUE(question_user_country_code) AS question_user_country_code,
  ANY_VALUE(question_topic) AS question_topic,
  MIN(question_sent) AS question_sent,
  ANY_VALUE(question_content) AS question_content
FROM farmer_questions
WHERE question_id IS NOT NULL
GROUP BY question_id;
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

It creates a saved view that takes all rows belonging to the same question, collapses them into one row, keeps the question's basic details, uses the earliest time the question was asked, and removes duplication caused by multiple responses.

```
con.execute("SHOW TABLES").df()
con.execute("SELECT COUNT(*) AS n_question_level FROM question_level;").df()
```

	n_question_level
0	5865819

```
con.execute("""
CREATE OR REPLACE VIEW challenge4_questions AS
SELECT
  question_id,
  question_language,
  question_user_country_code,
  question_topic,
  question_sent,

  question_content,

  LOWER(TRIM(REGEXP_REPLACE(question_content, '\\s+', ' ', 'g'))) AS cleaned_question,

  LOWER(TRIM(
    REGEXP_REPLACE(
      REGEXP_REPLACE(question_content, '[^a-zA-Z0-9\\s]', ' ', 'g'),
    '\\s+', ' ', 'g')
  )) AS normalized_question

FROM question_level
WHERE question_content IS NOT NULL AND TRIM(question_content) <> '';
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

It creates a saved view that keeps one row per question and adds cleaned and normalized versions of the question text so similar questions can be compared, grouped, and counted consistently.

```
con.execute("""
WITH q AS (
  SELECT
    question_id,
    normalized_question
  FROM challenge4_questions
  WHERE question_language = 'eng'
),
grp AS (
  SELECT
    normalized_question,
```

```
    COUNT(*) AS n_questions
  FROM q
  GROUP BY 1
)
SELECT
  COUNT(*) AS n_unique_normalized_questions,
  SUM(CASE WHEN n_questions > 1 THEN 1 ELSE 0 END) AS n_repeated_unique_questions,
  ROUND(100.0 * SUM(CASE WHEN n_questions > 1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS pct_unique_questions_that_repeat
FROM grp;
""").df()
```

	n_unique_normalized_questions	n_repeated_unique_questions	pct_unique_questions_that_repeat
0	2554315	177371.0	6.94

It checks how many unique English questions exist after normalization and what percentage of those questions were asked more than once.

This analysis shows that about 7% of normalized English questions are repeated, indicating a small but meaningful set of recurring farmer questions.

Repetition analysis was limited to English-language questions to ensure reliable text normalization, as extending this analysis to other languages would require language-specific preprocessing beyond the scope of this project.

```
con.execute("""
SHOW ALL TABLES;
""").df()
```

	database	schema	name	column_names	column_types	temporary
0	memory	main	challenge4_questions	[question_id, question_language, question_user...	[BIGINT, VARCHAR, VARCHAR, VARCHAR, TIMESTAMP ...	False
1	memory	main	farmer_questions	[question_id, question_user_id, question_langu...	[BIGINT, BIGINT, VARCHAR, VARCHAR, VARCHAR, TI...	False

```
con.execute("""
CREATE OR REPLACE VIEW challenge4_questions AS
SELECT
  question_id,
  question_language,
  question_user_country_code,
```

```
  question_topic,
  question_sent,
  question_content,

  LOWER(TRIM(REGEXP_REPLACE(question_content, '\\s+', ' ', 'g'))) AS cleaned_question,

  LOWER(TRIM(
    REGEXP_REPLACE(
      REGEXP_REPLACE(question_content, '[^a-zA-Z0-9\\s]', ' ', 'g'),
      '\\s+', ' ', 'g'
    )
  )) AS normalized_question
FROM question_level
WHERE question_content IS NOT NULL
  AND TRIM(question_content) <> '';
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

This step standardizes question text so repeated questions and shared themes can be identified reliably, which is required for Challenge 4 analysis.

```
con.execute("""
WITH q AS (
  SELECT question_id, normalized_question
  FROM challenge4_questions
  WHERE question_language = 'eng'
),
grp AS (
  SELECT normalized_question, COUNT(*) AS n_questions
  FROM q
  GROUP BY 1
),
flagged AS (
  SELECT
    q.question_id,
    CASE WHEN grp.n_questions > 1 THEN 1 ELSE 0 END AS is_repeated
  FROM q
  JOIN grp USING (normalized_question)
)
SELECT
  COUNT(*) AS n_english_questions,
  SUM(is_repeated) AS n_questions_in_repeated_groups,
  ROUND(100.0 * SUM(is_repeated) / COUNT(*), 2) AS pct_questions_in_repeated_groups
FROM flagged;
```

```
""").df()
```

	n_english_questions	n_questions_in_repeated_groups	pct_questions_in_repeated_groups
0	2940542	563598.0	19.17

Approximately 19% of English questions are repeats, indicating a substantial share of farmers ask questions that have already been asked before.

```
con.execute("""
SELECT
  LOWER(TRIM(question_topic)) AS topic,
  COUNT(DISTINCT question_id) AS n_questions
FROM question_level
WHERE question_topic IS NOT NULL
  AND TRIM(question_topic) <> ''
GROUP BY 1
ORDER BY n_questions DESC;
""").df()
```

	topic	n_questions
0	maize	535569
1	chicken	430894
2	cattle	425027
3	tomato	326011
4	poultry	272243
...	...	...
143	leucaena	17
144	setaria	15
145	mulberry	13
146	purple-vetch	7
147	cranberry	3

148 rows × 2 columns

It counts how many unique questions exist for each topic (like maize, chicken, cattle), ranks those topics from most to least asked about, and shows which crops or animals dominate farmer questions, counting each question only once.

Most farmer questions focus on a small number of crops and animals. Maize is the most common topic, with about 535,000 unique questions, followed by chicken with about 431,000 questions and cattle with about 425,000 questions. When combined with tomato at roughly 326,000 questions and poultry at about 272,000 questions, these five topics alone account for well over two million questions, showing that farmers mainly ask about a few key crops and livestock. At the same time, the dataset includes 148 different topics, many of which appear only rarely. Dozens of topics have fewer than 100 questions, and some appear fewer than 10 times, indicating a long tail of less common crops and animals. The topic labels also vary in how specific they are, with very specific topics such as maize, cassava, and goat appearing alongside broader labels like crop, animal, and livestock. This makes it important to clearly separate crop specific questions from general agricultural questions when answering the Challenge 4 questions.

```
con.execute("""
SELECT COUNT(DISTINCT LOWER(TRIM(question_topic))) AS n_distinct_topics
FROM question_level
WHERE question_topic IS NOT NULL
  AND TRIM(question_topic) <> '';
""").df()
```

	n_distinct_topics
0	148

```
con.execute("""
WITH topic_counts AS (
  SELECT
    LOWER(TRIM(question_topic)) AS topic,
    COUNT(DISTINCT question_id) AS n_questions
  FROM question_level
  WHERE question_topic IS NOT NULL
    AND TRIM(question_topic) <> ''
  GROUP BY 1
),
summary AS (
  SELECT
    COUNT(*) AS n_topics_total,
    SUM(CASE WHEN n_questions < 100 THEN 1 ELSE 0 END) AS n_topics_lt_100
  FROM topic_counts
)
SELECT
  n_topics_total,
  n_topics_lt_100,
  ROUND(100.0 * n_topics_lt_100 / n_topics_total, 2) AS pct_topics_lt_100
FROM summary;
""").df()
```

	n_topics_total	n_topics_lt_100	pct_topics_lt_100
0	148	19.0	12.84

```
top_n = 15

con.execute(f"""
WITH topic_counts AS (
  SELECT
    LOWER(TRIM(question_topic)) AS topic,
    COUNT(DISTINCT question_id) AS n_questions
  FROM question_level
  WHERE question_topic IS NOT NULL
    AND TRIM(question_topic) <> ''
  GROUP BY 1
```

```
),
ranked AS (
  SELECT
    topic,
    n_questions,
    ROW_NUMBER() OVER (ORDER BY n_questions DESC) AS rnk
  FROM topic_counts
)
SELECT
  {top_n} AS top_n,
  SUM(CASE WHEN rnk <= {top_n} THEN n_questions ELSE 0 END) AS questions_in_top_n,
  SUM(n_questions) AS questions_all_topics,
  ROUND(
    100.0 * SUM(CASE WHEN rnk <= {top_n} THEN n_questions ELSE 0 END) / SUM(n_questions),
    2
  ) AS pct_questions_in_top_n
FROM ranked;
""").df()
```

	top_n	questions_in_top_n	questions_all_topics	pct_questions_in_top_n
0	15	3078156.0	4193810.0	73.4

```
topic_counts = con.execute("""
SELECT
  LOWER(TRIM(question_topic)) AS topic,
  COUNT(DISTINCT question_id) AS n_questions
FROM question_level
WHERE question_topic IS NOT NULL
  AND TRIM(question_topic) <> ''
GROUP BY 1
ORDER BY n_questions DESC
""").df()

topic_counts
```

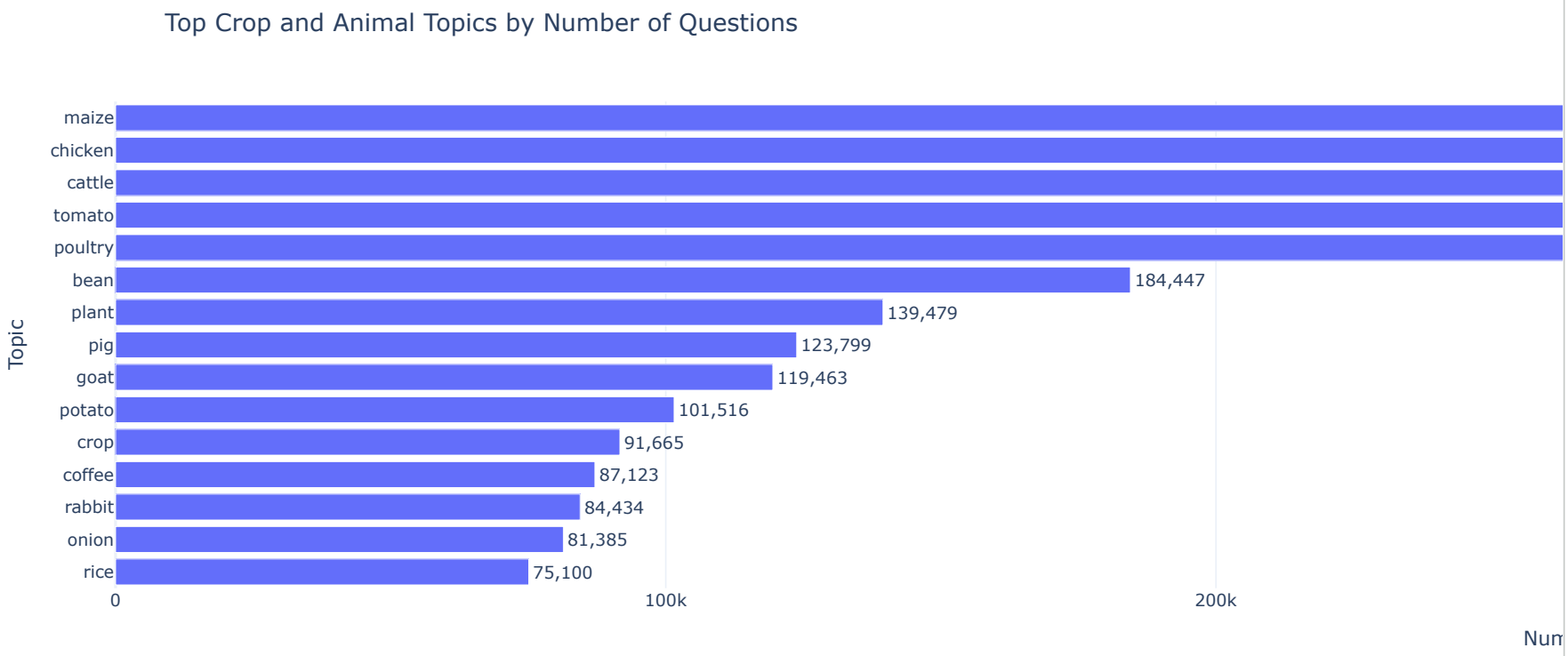
	topic	n_questions
0	maize	535567
1	chicken	430896
2	cattle	425028
3	tomato	326012
4	poultry	272242
...	...	...
143	leucaena	17
144	setaria	15
145	mulberry	13
146	purple-vetch	7
147	cranberry	3

148 rows × 2 columns

Next steps: Generate code with `topic_counts` New interactive sheet

```
import plotly.express as px

df = topic_counts.copy()

# Limit to top 15 topics
top_n = 15
df_top = df.sort_values("n_questions", ascending=False).head(top_n)

fig = px.bar(
    df_top,
    x="n_questions",
    y="topic",
    orientation="h",
    text="n_questions",
    title="Top Crop and Animal Topics by Number of Questions",
    labels={
        "n_questions": "Number of Unique Questions",
        "topic": "Topic"
    }
)
```

```
fig.update_traces(
    texttemplate="%{text:,}",
    textposition="outside"
)

fig.update_layout(
    template="plotly_white",
    yaxis=dict(categoryorder="total ascending"),
    xaxis_title="Number of Unique Questions",
    yaxis_title="Topic"
)

fig.show()
```




```
con.execute("""
CREATE OR REPLACE TABLE topic_classification AS
SELECT * FROM (
  VALUES
```

```
        -- Crop-Specific (named crops / plants / forages)
        ('maize','Crop-Specific'),
        ('tomato','Crop-Specific'),
        ('bean','Crop-Specific'),
        ('potato','Crop-Specific'),
        ('coffee','Crop-Specific'),
        ('onion','Crop-Specific'),
        ('rice','Crop-Specific'),
        ('banana','Crop-Specific'),
        ('cabbage','Crop-Specific'),
        ('cotton','Crop-Specific'),
        ('watermelon','Crop-Specific'),
        ('passion-fruit','Crop-Specific'),
        ('peanut','Crop-Specific'),
        ('kale','Crop-Specific'),
        ('cassava','Crop-Specific'),
        ('sugar-cane','Crop-Specific'),
        ('wheat','Crop-Specific'),
        ('millet','Crop-Specific'),
        ('tea','Crop-Specific'),
        ('carrot','Crop-Specific'),
        ('plantain','Crop-Specific'),
        ('sunflower','Crop-Specific'),
        ('avocado','Crop-Specific'),
        ('capsicum','Crop-Specific'),
        ('sweet-potato','Crop-Specific'),
        ('mango','Crop-Specific'),
        ('melon','Crop-Specific'),
        ('spinach','Crop-Specific'),
        ('pea','Crop-Specific'),
        ('orange','Crop-Specific'),
        ('chilli','Crop-Specific'),
        ('ginger','Crop-Specific'),
        ('mushroom','Crop-Specific'),
        ('soya','Crop-Specific'),
        ('pineapple','Crop-Specific'),
        ('aubergine','Crop-Specific'),
        ('pumpkin','Crop-Specific'),
        ('cowpea','Crop-Specific'),
        ('paw-paw','Crop-Specific'),
        ('cashew-nut','Crop-Specific'),
        ('tobacco','Crop-Specific'),
        ('apple','Crop-Specific'),
        ('olive','Crop-Specific'),
        ('garlic','Crop-Specific'),
        ('butternut-squash','Crop-Specific'),
        ('yam','Crop-Specific'),
        ('cocoa','Crop-Specific'),
        ('taro','Crop-Specific'),
```

```
        ('african-nightshade','Crop-Specific'),
        ('mung-bean','Crop-Specific'),
        ('pigeon-pea','Crop-Specific'),
        ('eucalyptus','Crop-Specific'),
        ('beetroot','Crop-Specific'),
        ('coconut','Crop-Specific'),
        ('sisal','Crop-Specific'),
        ('french-bean','Crop-Specific'),
        ('miraa','Crop-Specific'),
        ('guava','Crop-Specific'),
        ('macadamia','Crop-Specific'),
        ('lemon','Crop-Specific'),
        ('pyrethrum','Crop-Specific'),
        ('pear','Crop-Specific'),
        ('squash','Crop-Specific'),
        ('strawberry','Crop-Specific'),
        ('parsley','Crop-Specific'),
        ('cucumber','Crop-Specific'),
        ('snow-pea','Crop-Specific'),
        ('chia','Crop-Specific'),
        ('jackfruit','Crop-Specific'),
        ('okra','Crop-Specific'),
        ('rapeseed','Crop-Specific'),
        ('finger-millet','Crop-Specific'),
        ('boma-rhodes','Crop-Specific'),
        ('safflower','Crop-Specific'),
        ('clover','Crop-Specific'),
        ('bamboo','Crop-Specific'),
        ('acacia','Crop-Specific'),
        ('amaranth','Crop-Specific'),
        ('grape','Crop-Specific'),
        ('peach','Crop-Specific'),
        ('desmodium','Crop-Specific'),
        ('nightshade','Crop-Specific'),
        ('radish','Crop-Specific'),
        ('barley','Crop-Specific'),
        ('cyprus','Crop-Specific'),
        ('lucern','Crop-Specific'),
        ('brachiaria-grass','Crop-Specific'),
        ('sesame','Crop-Specific'),
        ('oat','Crop-Specific'),
        ('coriander','Crop-Specific'),
        ('flax','Crop-Specific'),
        ('courgette','Crop-Specific'),
        ('black-nightshade','Crop-Specific'),
        ('broccoli','Crop-Specific'),
        ('gooseberry','Crop-Specific'),
        ('cauliflower','Crop-Specific'),
        ('lettuce','Crop-Specific'),
```

```
        ('chard','Crop-Specific'),
        ('sudan-grass','Crop-Specific'),
        ('chickpea','Crop-Specific'),
        ('leek','Crop-Specific'),
        ('asparagus','Crop-Specific'),
        ('celery','Crop-Specific'),
        ('caliandra','Crop-Specific'),
        ('snap-pea','Crop-Specific'),
        ('castor-bean','Crop-Specific'),
        ('apricot','Crop-Specific'),
        ('lupin','Crop-Specific'),
        ('rye','Crop-Specific'),
        ('vetch','Crop-Specific'),
        ('blackberry','Crop-Specific'),
        ('leucaena','Crop-Specific'),
        ('setaria','Crop-Specific'),
        ('mulberry','Crop-Specific'),
        ('purple-vetch','Crop-Specific'),
        ('cranberry','Crop-Specific'),
        ('collard-greens','Crop-Specific'),
        ('napier-grass','Crop-Specific'),  -- forage crop

        -- Livestock-Specific (named animals)
        ('chicken','Livestock-Specific'),
        ('cattle','Livestock-Specific'),
        ('poultry','Livestock-Specific'),
        ('camel','Livestock-Specific'),
        ('tilapia','Livestock-Specific'),
        ('guinea-pig','Livestock-Specific'),
        ('pig','Livestock-Specific'),
        ('goat','Livestock-Specific'),
        ('rabbit','Livestock-Specific'),
        ('sheep','Livestock-Specific'),
        ('fish','Livestock-Specific'),
        ('duck','Livestock-Specific'),
        ('dog','Livestock-Specific'),
        ('bird','Livestock-Specific'),
        ('pigeon','Livestock-Specific'),
        ('guinea-fowl','Livestock-Specific'),
        ('cat','Livestock-Specific'),
        ('turkey','Livestock-Specific'),
        ('ostrich','Livestock-Specific'),

        -- General (crop-independent / cross-cutting categories)
        ('plant','General'),
        ('crop','General'),
        ('animal','General'),
        ('livestock','General'),
        ('tree','General'),
```

```
    ('vegetable','General'),
    ('grass','General'),
    ('cereal','General'),
    ('greens','General'),
    ('bee','General'),
    ('locust','General')   -- pest affects many crops
) AS t(question_topic, question_type);
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

It creates a table that maps each question_topic to one of three categories: Crop Specific for named crops and plants, Livestock Specific for named animals, and General for broad or cross cutting topics like crop, animal, or livestock.

```
con.execute("""
CREATE OR REPLACE VIEW challenge4_labeled AS
SELECT
  q.question_id,
  q.question_language,
  LOWER(TRIM(q.question_topic)) AS question_topic,
  q.question_user_country_code,
  q.question_sent,
  COALESCE(tc.question_type, 'Unknown') AS question_type
FROM question_level q
LEFT JOIN topic_classification tc
  ON LOWER(TRIM(q.question_topic)) = tc.question_topic
WHERE q.question_id IS NOT NULL
  AND q.question_topic IS NOT NULL
  AND TRIM(q.question_topic) <> '';
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

This labels each question as Crop Specific, Livestock Specific, or General by matching its topic to your classification table, and marks it as “Unknown” if no match is found.

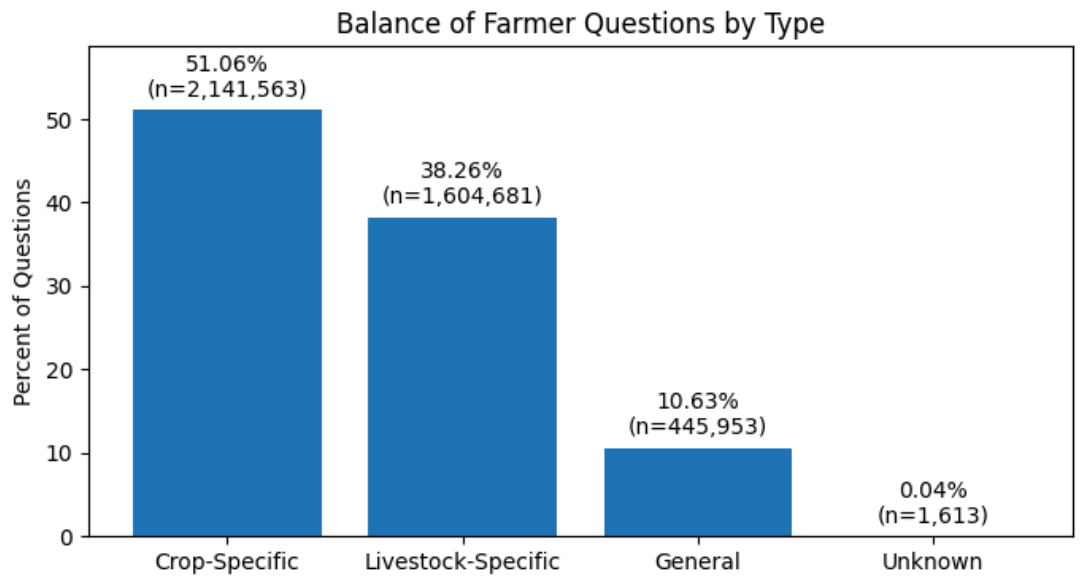
```
con.execute("""
SELECT
  question_type,
  COUNT(DISTINCT question_id) AS n_questions,
  ROUND(
    100.0 * COUNT(DISTINCT question_id)
    / SUM(COUNT(DISTINCT question_id)) OVER (),
    2
```

```
  ) AS pct
FROM challenge4_labeled
GROUP BY 1
ORDER BY n_questions DESC;
""").df()
```

	question_type	n_questions	pct
0	Crop-Specific	2141561	51.06
1	Livestock-Specific	1604680	38.26
2	General	445956	10.63
3	Unknown	1613	0.04

this query counts how many unique questions fall into each question type and calculates what percentage of all questions each type represents.

```
import pandas as pd
import matplotlib.pyplot as plt

balance_df = pd.DataFrame({
    "category": ["Crop-Specific", "Livestock-Specific", "General", "Unknown"],
    "pct": [51.06, 38.26, 10.63, 0.04],
    "n_questions": [2141563, 1604681, 445953, 1613]
})

plt.figure(figsize=(8, 4))
bars = plt.bar(balance_df["category"], balance_df["pct"])
plt.title("Balance of Farmer Questions by Type")
plt.ylabel("Percent of Questions")
plt.ylim(0, max(balance_df["pct"]) * 1.15)

# Label each bar with percent and count
for bar, pct, n in zip(bars, balance_df["pct"], balance_df["n_questions"]):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + (max(balance_df["pct"]) * 0.02),
        f"{pct:.2f}%\n(n={n:,})",
        ha="center",
        va="bottom"
    )

plt.show()
```



The results show that most farmer questions are highly specific rather than general. Crop specific questions make up about 51 percent of all questions, with roughly 2.14 million unique questions, making this the largest category. Livestock specific questions account for about 38 percent, or approximately 1.60 million questions, showing that animal related concerns are also a major focus for farmers. In contrast, general agricultural questions represent only about 11 percent, or around 446,000 questions, indicating that relatively few farmers ask broad, crop independent questions. The Unknown category is negligible, with only 1,613 questions or about 0.04 percent, which suggests that the topic classification covers nearly the entire dataset. Overall, this distribution indicates that farmers primarily seek targeted, crop or animal specific advice, rather than general farming guidance, directly supporting the core goal of Challenge 4 to understand the balance between specific and general questions.

```
con.execute("""
SELECT
  question_user_country_code,
  question_type,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_labeled
GROUP BY 1,2
ORDER BY 3 DESC;
""").df()
```

	question_user_country_code	question_type	n_questions
0	ke	Livestock-Specific	838952
1	ke	Crop-Specific	776818
2	ug	Crop-Specific	718095
3	tz	Crop-Specific	646604
4	ug	Livestock-Specific	451428
5	tz	Livestock-Specific	314292
6	ke	General	237484
7	ug	General	155943
8	tz	General	52520
9	ke	Unknown	1557
10	ug	Unknown	56
11	gb	Crop-Specific	43
12	gb	Livestock-Specific	11
13	gb	General	7

It counts how many crop specific, livestock specific, general, and unknown questions were asked in each country and shows which question types are most common by location.

```
con.execute("""
SELECT
  LOWER(TRIM(question_topic)) AS topic,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_labeled
WHERE question_type = 'Unknown'
  AND question_topic IS NOT NULL
  AND TRIM(question_topic) <> ''
GROUP BY 1
ORDER BY n_questions DESC
LIMIT 50;
""").df()
```

	topic	n_questions
0	macademia	1327
1	corriander	286

It lists the most common topics among questions that could not be classified, showing which topic names are driving the small “Unknown” category.

```
con.execute("""
CREATE OR REPLACE TABLE topic_classification AS
SELECT DISTINCT question_topic, question_type
FROM topic_classification;
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

```
country_pct = con.execute("""
SELECT
  question_user_country_code AS country,
  CASE
    WHEN question_type IN ('Crop-Specific','Livestock-Specific') THEN 'Specific'
    WHEN question_type = 'General' THEN 'General'
    ELSE 'Unknown'
  END AS bucket,
  COUNT(DISTINCT question_id) AS n_questions,
  ROUND(
    100.0 * COUNT(DISTINCT question_id)
    / SUM(COUNT(DISTINCT question_id)) OVER (PARTITION BY question_user_country_code),
    2
  ) AS pct_within_country
FROM challenge4_labeled
GROUP BY 1,2
ORDER BY country, pct_within_country DESC;
""").df()

country_pct
```

	country	bucket	n_questions	pct_within_country
0	gb	Specific	54	88.52
1	gb	General	7	11.48
2	ke	Specific	1615770	87.11
3	ke	General	237484	12.80
4	ke	Unknown	1557	0.08
5	tz	Specific	960896	94.82
6	tz	General	52520	5.18
7	ug	Specific	1169523	88.23
8	ug	General	155943	11.76
9	ug	Unknown	56	0.00

Next steps: Generate code with `country_pct` New interactive sheet

```
month_pct = con.execute("""
SELECT
  DATE_TRUNC('month', question_sent) AS month,
  CASE
    WHEN question_type IN ('Crop-Specific','Livestock-Specific') THEN 'Specific'
    WHEN question_type = 'General' THEN 'General'
    ELSE 'Unknown'
  END AS bucket,
  COUNT(DISTINCT question_id) AS n_questions,
  ROUND(
    100.0 * COUNT(DISTINCT question_id)
    / SUM(COUNT(DISTINCT question_id)) OVER (PARTITION BY DATE_TRUNC('month', question_sent)),
    2
  ) AS pct_within_month
FROM challenge4_labeled
WHERE question_sent IS NOT NULL
GROUP BY 1,2
ORDER BY month, bucket;
""").df()

month_pct.head(20)
```

	month	bucket	n_questions	pct_within_month
0	2017-11-01 00:00:00+00:00	General	1115	9.95
1	2017-11-01 00:00:00+00:00	Specific	10089	90.05
2	2017-12-01 00:00:00+00:00	General	2457	9.80
3	2017-12-01 00:00:00+00:00	Specific	22615	90.16
4	2017-12-01 00:00:00+00:00	Unknown	12	0.05
5	2018-01-01 00:00:00+00:00	General	2678	8.67
6	2018-01-01 00:00:00+00:00	Specific	28212	91.29
7	2018-01-01 00:00:00+00:00	Unknown	14	0.05
8	2018-02-01 00:00:00+00:00	General	2614	9.40
9	2018-02-01 00:00:00+00:00	Specific	25173	90.55
10	2018-02-01 00:00:00+00:00	Unknown	14	0.05
11	2018-03-01 00:00:00+00:00	General	5321	12.93
12	2018-03-01 00:00:00+00:00	Specific	35802	86.97
13	2018-03-01 00:00:00+00:00	Unknown	45	0.11
14	2018-04-01 00:00:00+00:00	General	5338	10.97
15	2018-04-01 00:00:00+00:00	Specific	43313	88.99
16	2018-04-01 00:00:00+00:00	Unknown	22	0.05
17	2018-05-01 00:00:00+00:00	General	5666	11.17
18	2018-05-01 00:00:00+00:00	Specific	45036	88.78
19	2018-05-01 00:00:00+00:00	Unknown	24	0.05

Next steps: Generate code with `month_pct` New interactive sheet

It shows, for each month, how many unique questions were asked and what percentage of that month's questions were specific (crop or livestock), general, or unknown, so you can see how question types change over time.

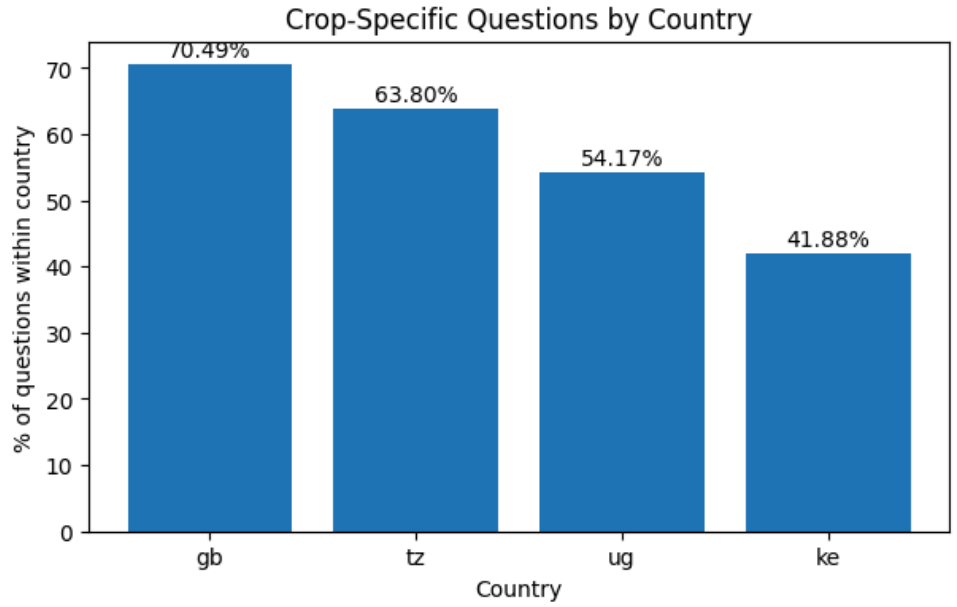
```
country_type_pct = con.execute("""
WITH country_totals AS (
  SELECT
```

```
    question_user_country_code AS country,
    COUNT(DISTINCT question_id) AS total_questions
  FROM challenge4_labeled
  GROUP BY 1
),
country_type_counts AS (
  SELECT
    question_user_country_code AS country,
    question_type,
    COUNT(DISTINCT question_id) AS n_questions
  FROM challenge4_labeled
  GROUP BY 1, 2
)
SELECT
  c.country,
  c.question_type,
  c.n_questions,
  ROUND(100.0 * c.n_questions / t.total_questions, 2) AS pct_within_country
FROM country_type_counts c
JOIN country_totals t
  ON c.country = t.country
ORDER BY c.country, pct_within_country DESC;
""").df()
```

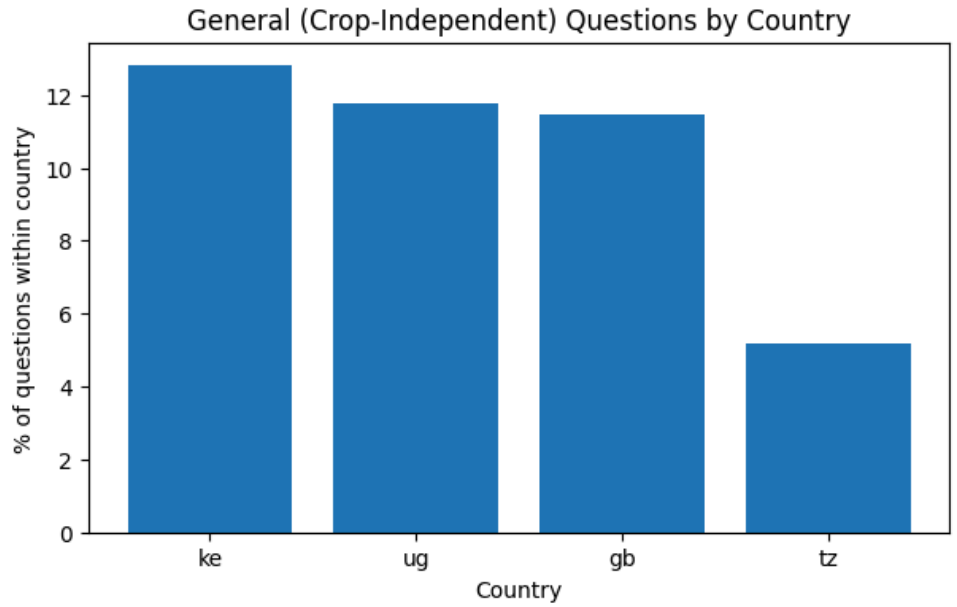
It calculates, for each country, how many questions fall into each question type and what percentage of that country's total questions each type represents.

```
import matplotlib.pyplot as plt

plot_country = (
    country_type_pct[country_type_pct["question_type"] == "Crop-Specific"]
    .sort_values("pct_within_country", ascending=False)
)

plt.figure(figsize=(7, 4))
bars = plt.bar(plot_country["country"], plot_country["pct_within_country"])
plt.title("Crop-Specific Questions by Country")
plt.xlabel("Country")
plt.ylabel("% of questions within country")

for bar, pct in zip(bars, plot_country["pct_within_country"]):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
```

```
        bar.get_height() + 0.5,
        f"{pct:.2f}%",
        ha="center",
        va="bottom"
    )

plt.show()
```




```
import matplotlib.pyplot as plt

plot_country = (
    country_pct[country_pct["bucket"] == "General"]
    .sort_values("pct_within_country", ascending=False)
)

plt.figure(figsize=(7,4))
plt.bar(plot_country["country"], plot_country["pct_within_country"])
plt.title("General (Crop-Independent) Questions by Country")
plt.xlabel("Country")
plt.ylabel("% of questions within country")
plt.show()
```




```
con.execute("""
SELECT
  question_type,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_labeled
GROUP BY 1
ORDER BY n_questions DESC;
""").df()
```

	question_type	n_questions
0	Crop-Specific	2141562
1	Livestock-Specific	1604683
2	General	445952
3	Unknown	1613

It counts how many unique questions fall into each question type and ranks them from most common to least common.

```
con.execute("""
CREATE OR REPLACE VIEW challenge4_time AS
```

```
SELECT
  question_id,
  question_type,
  question_user_country_code,
  DATE_TRUNC('month', question_sent) AS question_month
FROM challenge4_labeled
WHERE question_sent IS NOT NULL;
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

It creates a view that keeps each question's type and country and adds a new column showing the month the question was asked, so questions can be analyzed over time.

```
time_df = con.execute("""
SELECT
  question_month,
  question_type,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_time
GROUP BY 1, 2
ORDER BY question_month, question_type
""").df()
```

It counts how many unique questions of each question type were asked in each month and orders the results by time.

```
import pandas as pd
import matplotlib.pyplot as plt

time_counts = con.execute("""
SELECT
  question_month,
  question_type,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_time
WHERE question_type IN ('Crop-Specific','Livestock-Specific','General')
GROUP BY 1,2
ORDER BY 1,2;
""").df()
```

It queries the time-based view to count how many distinct questions were asked each month for each question type (Crop-Specific, Livestock-Specific, and General), excludes Unknowns, and returns the results ordered by month and type so they can be plotted over time.

```
wide = (
    time_counts
    .pivot(index="question_month", columns="question_type", values="n_questions")
    .fillna(0)
    .sort_index()
)

wide.head()
```

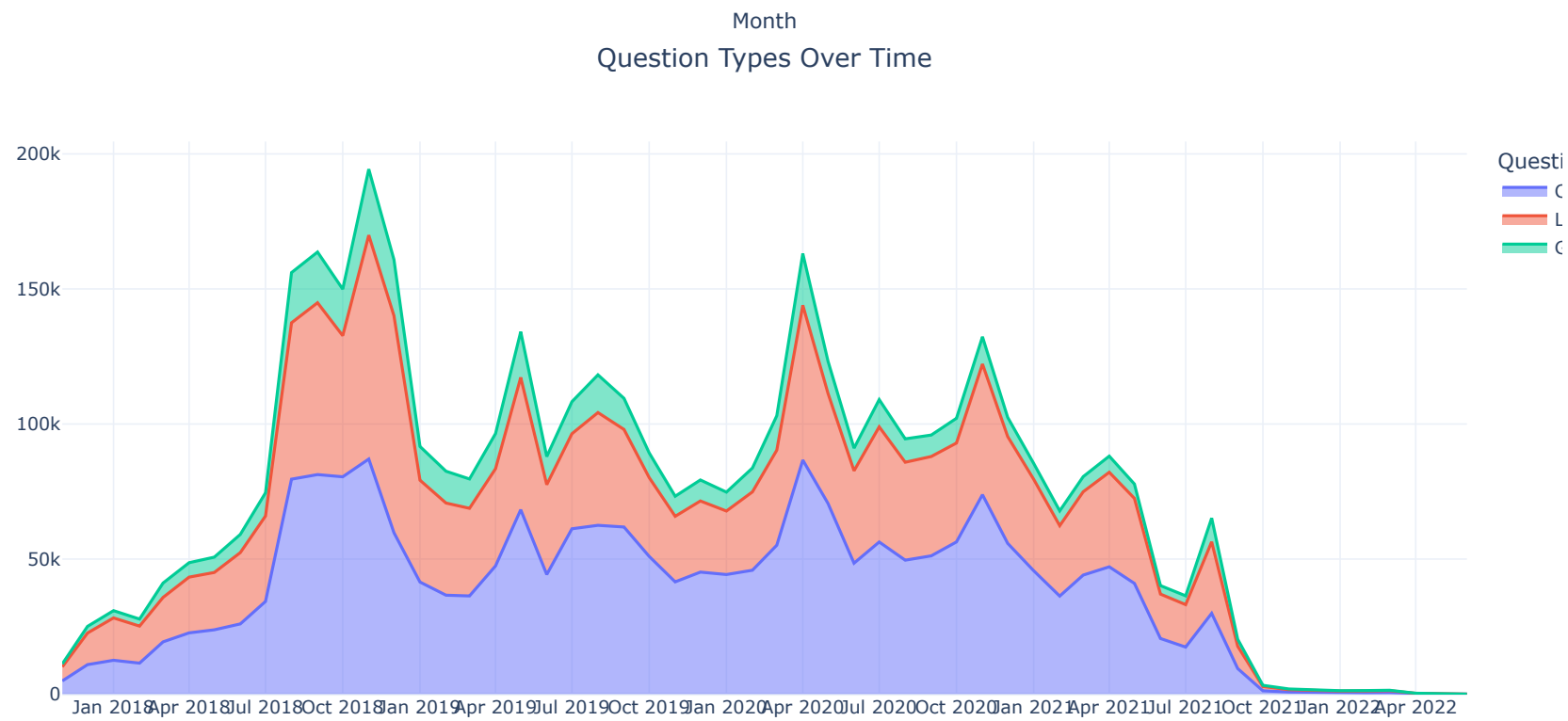
question_month	question_type	Crop-Specific	General	Livestock-Specific
2017-11-01 00:00:00+00:00		4885.0	1115.0	5204.0
2017-12-01 00:00:00+00:00		10911.0	2457.0	11704.0
2018-01-01 00:00:00+00:00		12524.0	2678.0	15688.0
2018-02-01 00:00:00+00:00		11466.0	2614.0	13707.0
2018-03-01 00:00:00+00:00		19343.0	5321.0	16459.0

Next steps: Generate code with `wide` New interactive sheet

This code reshapes the monthly results into a wide table where each row is a month, each column is a question type (Crop-Specific, General, Livestock-Specific), and the values are the number of distinct questions, filling missing combinations with zeros and sorting by time so the data is ready for time-series charts.

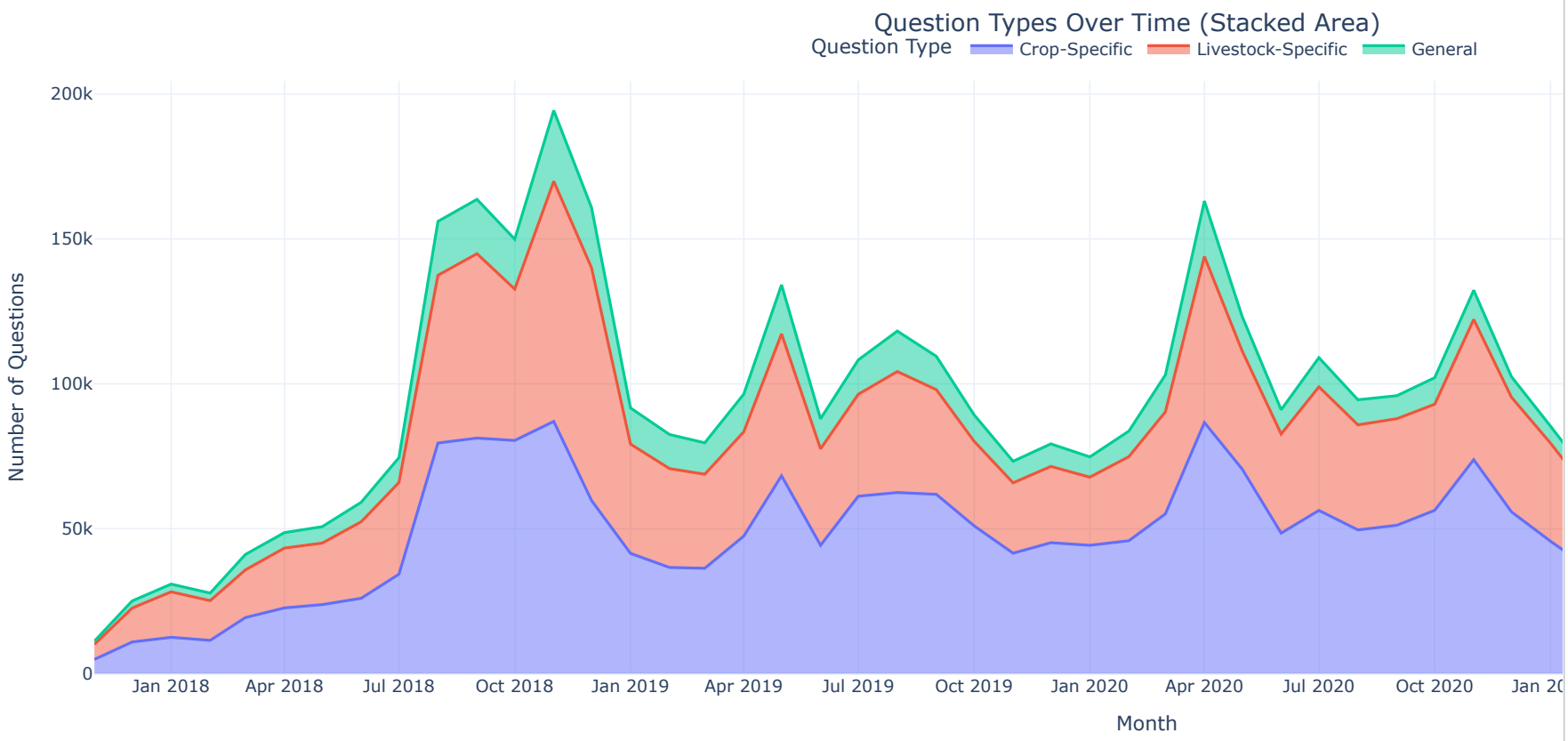
```
import plotly.express as px
import pandas as pd

plot_df = wide.reset_index().rename(columns={"question_month": "Month"})

# Ensure Month is datetime
plot_df["Month"] = pd.to_datetime(plot_df["Month"])
```

```
# Ensure required columns exist, fill missing with 0
needed = ["Crop-Specific", "Livestock-Specific", "General"]
for col in needed:
    if col not in plot_df.columns:
        plot_df[col] = 0

fig = px.area(
    plot_df.sort_values("Month"),
    x="Month",
    y=needed,
    title="Question Types Over Time",
    labels={"value": "Number of Questions", "variable": "Question Type"}
)

fig.update_layout(
    template="plotly_white",
    hovermode="x unified",
    title_x=0.5,
    width=1100,
    height=550,
    legend_title_text="Question Type"
)

fig.update_xaxes(dtick="M3", tickformat="%b %Y")

fig.show()
```



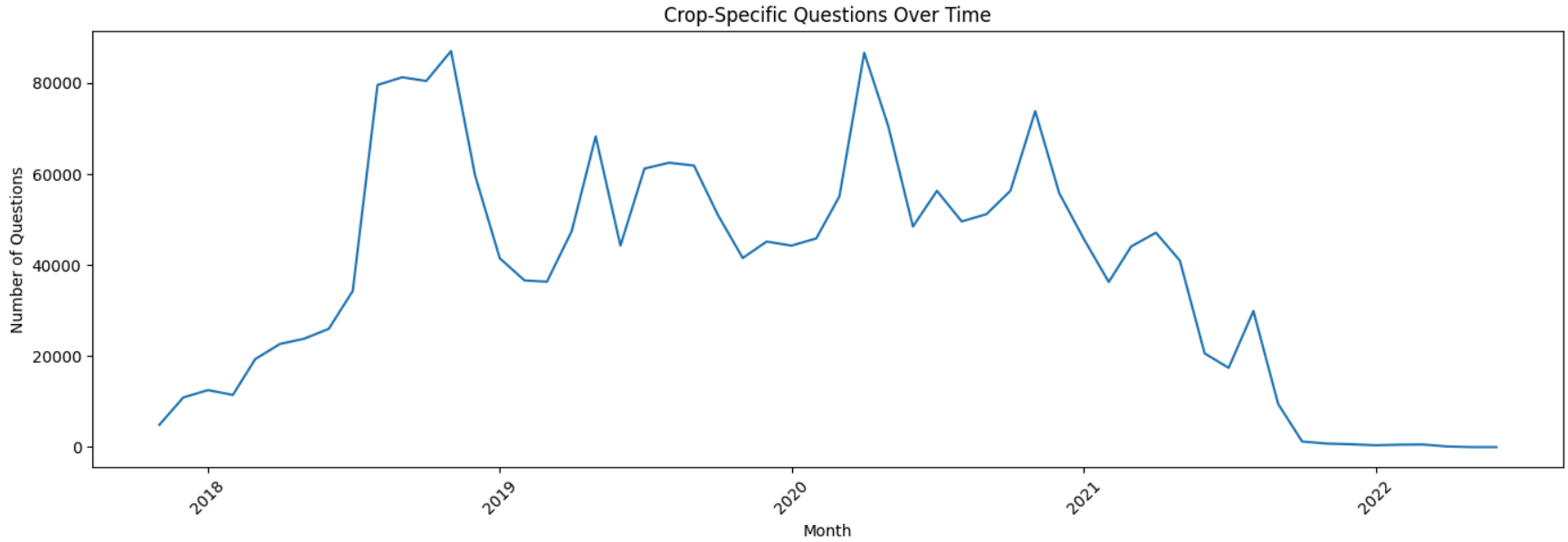

```
import plotly.express as px

plot_df = wide.reset_index()
x_col = plot_df.columns[0]

fig = px.area(
    plot_df,
    x=x_col,
    y=["Crop-Specific", "Livestock-Specific", "General"],
    title="Question Types Over Time (Stacked Area)",
    labels={x_col: "Month", "value": "Number of Questions", "variable": "Question Type"}
)

fig.update_layout(
    template="plotly_white",
    hovermode="x unified",
    title_x=0.5,
    width=1600,
```

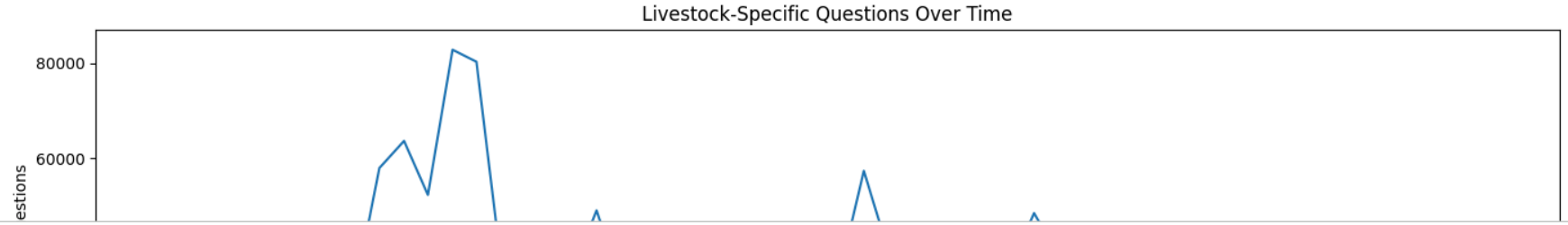
```
    height=550,
    margin=dict(l=60, r=40, t=70, b=60),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="center", x=0.5)
)

fig.update_xaxes(
    dtick="M3",
    tickformat="%b %Y"
)

fig.show()
```




```
import matplotlib.pyplot as plt
import pandas as pd
```

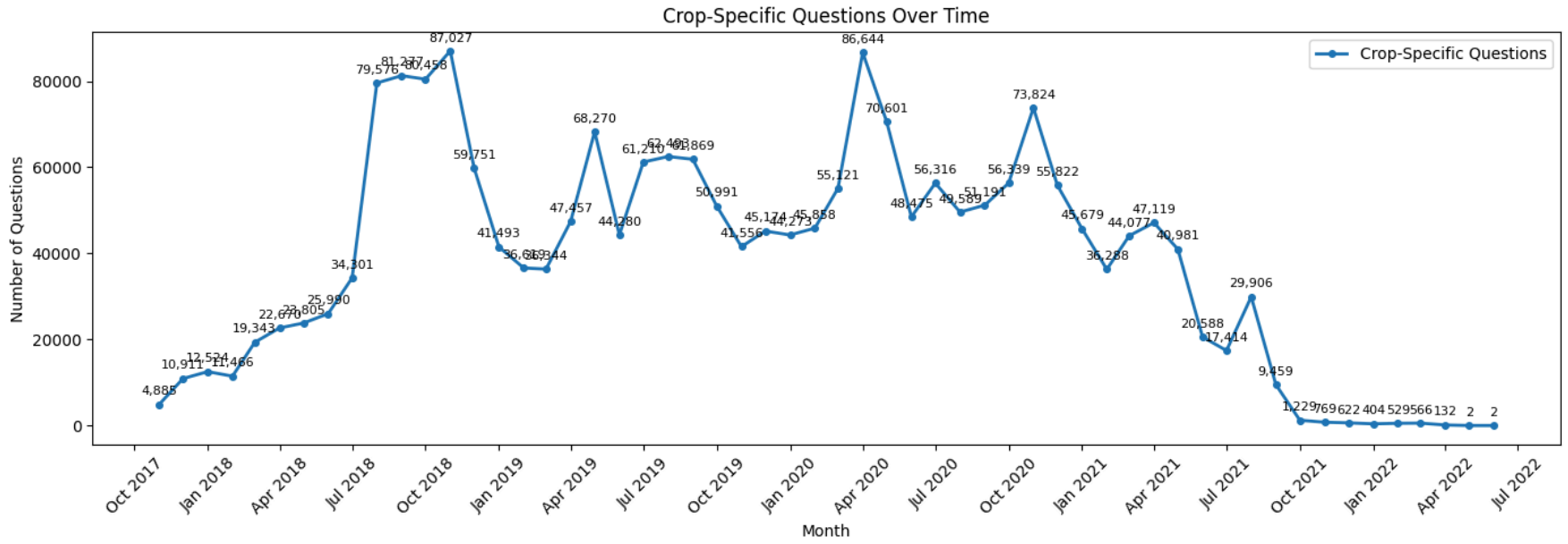
```
# Reset index so Month is a column
plot_df = wide.reset_index()
x_col = plot_df.columns[0]  # Month column

# Ensure Month is datetime (important for formatting)
plot_df[x_col] = pd.to_datetime(plot_df[x_col])

# ---------- 1. Crop-Specific ----------
plt.figure(figsize=(14, 5))
plt.plot(plot_df[x_col], plot_df["Crop-Specific"])
plt.title("Crop-Specific Questions Over Time")
plt.xlabel("Month")
plt.ylabel("Number of Questions")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# ---------- 2. Livestock-Specific ----------
plt.figure(figsize=(14, 5))
plt.plot(plot_df[x_col], plot_df["Livestock-Specific"])
plt.title("Livestock-Specific Questions Over Time")
plt.xlabel("Month")
plt.ylabel("Number of Questions")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# ---------- 3. General ----------
plt.figure(figsize=(14, 5))
plt.plot(plot_df[x_col], plot_df["General"])
plt.title("General Questions Over Time")
plt.xlabel("Month")
plt.ylabel("Number of Questions")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```





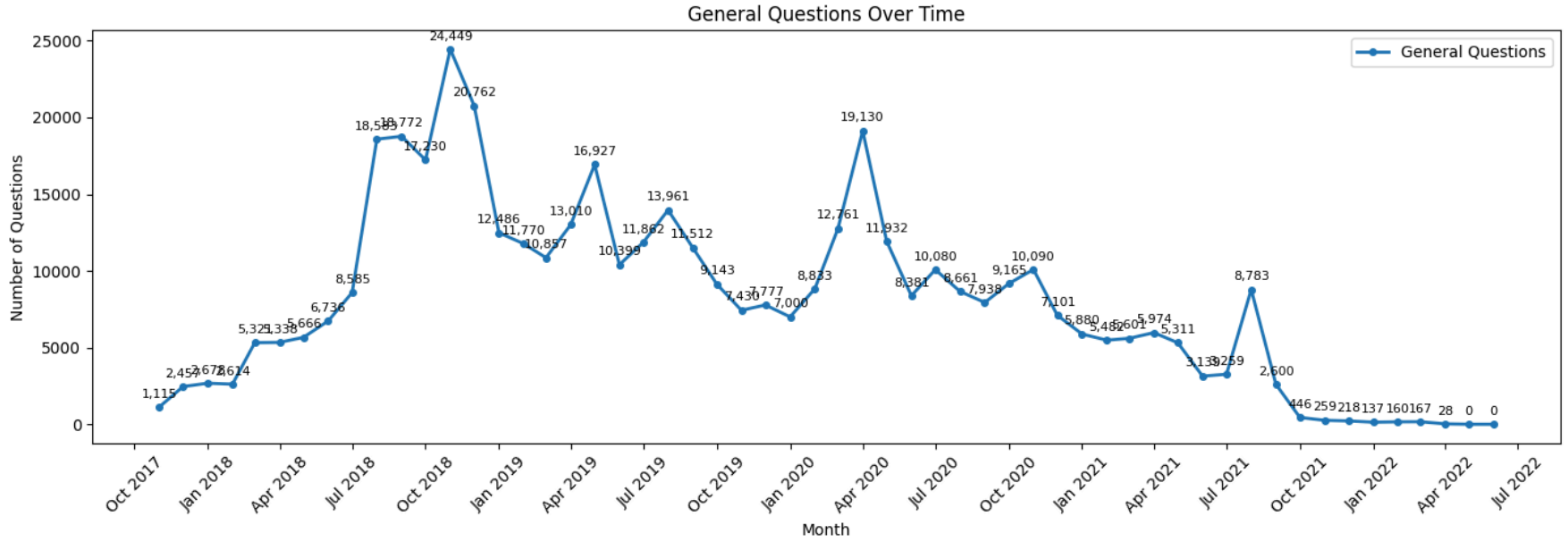

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import pandas as pd

# If Month is in the index, turn it into a column
plot_df = wide.reset_index()

# Use the first column as the x-axis (whatever it is named)
x_col = plot_df.columns[0]

# Make sure it's datetime
plot_df[x_col] = pd.to_datetime(plot_df[x_col])

plt.figure(figsize=(14, 5))
plt.plot(
    plot_df[x_col],
    plot_df["Crop-Specific"],
    marker="o",
    markersize=4,
```

```
        linewidth=2,
        label="Crop-Specific Questions"
    )

    for x, y in zip(plot_df[x_col], plot_df["Crop-Specific"]):
        plt.annotate(
            f"{int(y):,}",
            (x, y),
            textcoords="offset points",
            xytext=(0, 6),
            ha="center",
            fontsize=8
        )

    plt.title("Crop-Specific Questions Over Time")
    plt.xlabel("Month")
    plt.ylabel("Number of Questions")

    plt.gca().xaxis.set_major_locator(mdates.MonthLocator(interval=3))
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))

    plt.legend()
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```




```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

plt.figure(figsize=(14, 5))
plt.plot(
    plot_df[x_col],
    plot_df["General"],
    marker="o",
    markersize=4,
    linewidth=2,
    label="General Questions"
)

# Add value labels above each point
for x, y in zip(plot_df[x_col], plot_df["General"]):
    plt.annotate(
        f"{int(y):,}",          # format with commas
        (x, y),
        textcoords="offset points",
        xytext=(0, 6),          # slight vertical offset
        ha="center",
```

```
        fontsize=8
    )

plt.title("General Questions Over Time")
plt.xlabel("Month")
plt.ylabel("Number of Questions")

plt.gca().xaxis.set_major_locator(mdates.MonthLocator(interval=3))
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))

plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




```
general_topics = con.execute("""
SELECT
  question_topic,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_labeled
```

```
WHERE question_type = 'General'
GROUP BY question_topic
ORDER BY n_questions DESC
""").df()

general_topics.head(10)
```

	question_topic	n_questions
0	plant	139479
1	crop	91664
2	animal	55785
3	livestock	34990
4	tree	31073
5	vegetable	30094
6	bee	29070
7	grass	17346
8	cereal	14781
9	greens	1348

Next steps: Generate code with general_topics | New interactive sheet

This code counts how many unique questions fall under each General topic, using distinct question_ids, and then ranks those general topics from most common to least common so you can see which broad, crop-independent themes like plant, crop, or animal dominate General questions.

```
general_topics_refined = con.execute("""
SELECT
  question_topic,
  COUNT(DISTINCT question_id) AS n_questions
FROM challenge4_labeled
WHERE question_type = 'General'
  AND question_topic NOT IN (
    'plant', 'crop', 'animal', 'livestock',
    'tree', 'vegetable', 'cereal', 'greens'
  )
GROUP BY question_topic
```

```
ORDER BY n_questions DESC
""").df()

general_topics_refined.head(10)
```

	question_topic	n_questions
0	bee	29070
1	grass	17346
2	locust	325

Next steps: Generate code with general_topics_refined | New interactive sheet

This query filters General questions to remove very broad umbrella topics like plant, crop, and animal, then counts and ranks the remaining more specific general topics such as bee, grass, and locust by the number of unique questions, so you can see which cross-cutting but more concrete issues dominate within General questions.

```
cross_cutting = con.execute("""
WITH general AS (
  SELECT
    l.question_id,
    LOWER(f.question_content) AS q,
    l.question_user_country_code,
    l.question_sent
  FROM challenge4_labeled l
  JOIN farmer_questions f
    ON l.question_id = f.question_id
  WHERE l.question_type = 'General'
),
tagged AS (
  SELECT
    question_id,
    CASE
      WHEN q LIKE '%pest%' OR q LIKE '%insect%' OR q LIKE '%worm%' OR q LIKE '%aphid%' OR q LIKE '%locust%' THEN 'pests'
      WHEN q LIKE '%soil%' OR q LIKE '%compost%' OR q LIKE '%manure%' THEN 'soil'
      WHEN q LIKE '%plant%' OR q LIKE '%sow%' OR q LIKE '%seed%' OR q LIKE '%germinat%' THEN 'planting'
      WHEN q LIKE '%fertili%' OR q LIKE '%urea%' OR q LIKE '%npk%' THEN 'fertilizer'
      WHEN q LIKE '%disease%' OR q LIKE '%fung%' OR q LIKE '%rot%' OR q LIKE '%blight%' THEN 'disease'
      WHEN q LIKE '%rain%' OR q LIKE '%weather%' OR q LIKE '%drought%' OR q LIKE '%flood%' THEN 'weather'
      ELSE 'other'
    END AS theme
  FROM general
```

```
)
SELECT
  theme,
  COUNT(DISTINCT question_id) AS n_questions,
  ROUND(
    100.0 * COUNT(DISTINCT question_id)
    / SUM(COUNT(DISTINCT question_id)) OVER (),
    2
  ) AS pct_of_general
FROM tagged
GROUP BY theme
ORDER BY n_questions DESC;
""").df()

cross_cutting
```

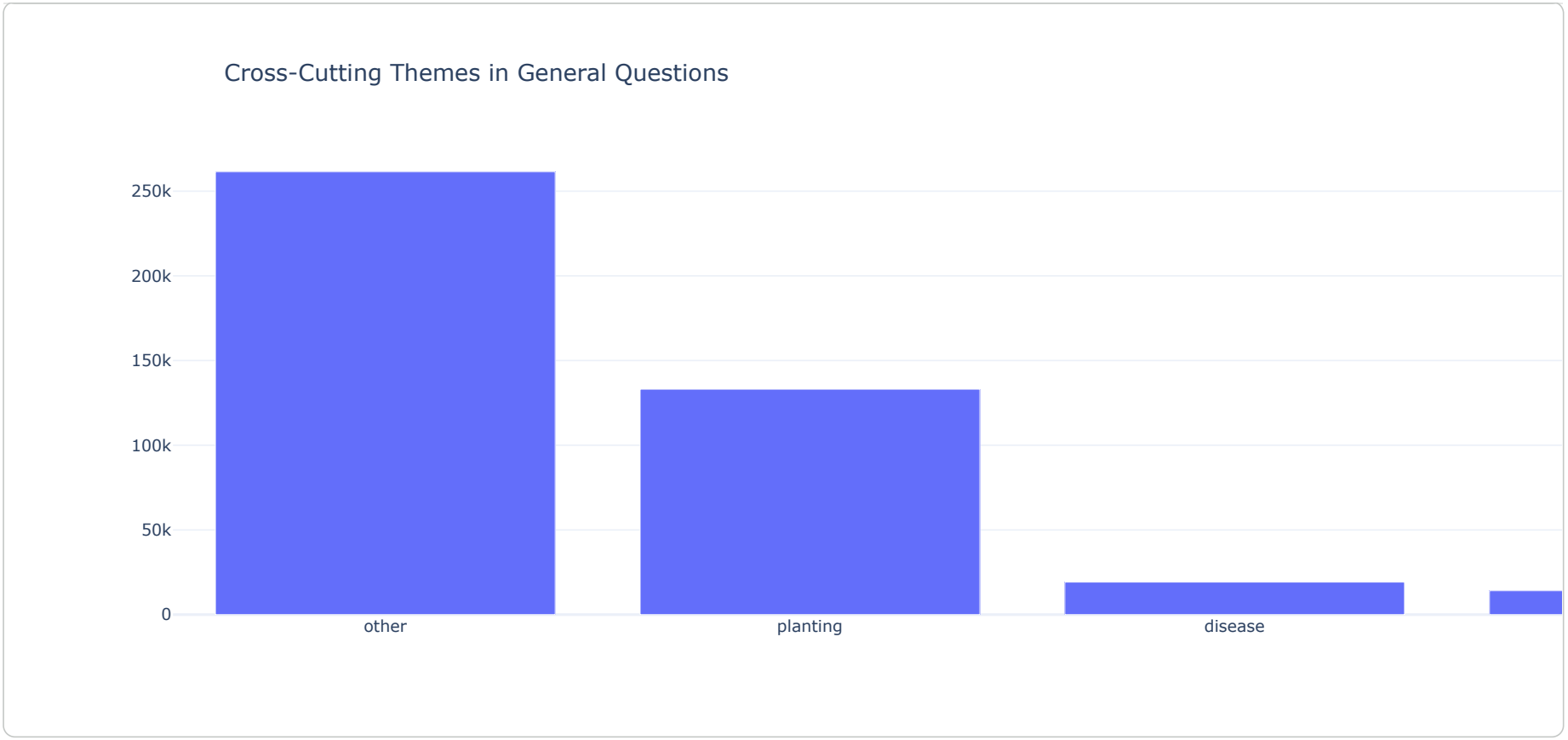
	theme	n_questions	pct_of_general
0	other	261431	58.62
1	planting	133087	29.84
2	disease	19172	4.30
3	pests	14099	3.16
4	soil	10604	2.38
5	fertilizer	4236	0.95
6	weather	3326	0.75

Next steps: Generate code with `cross_cutting` New interactive sheet

This query takes all General questions, scans their text for keywords, assigns each question to a cross-cutting theme like planting, pests, soil, disease, fertilizer, weather, or other, then counts how many unique questions fall into each theme and calculates what percentage of all General questions each theme represents.

```
import plotly.express as px

df = cross_cutting.copy()

fig = px.bar(
    df.sort_values("n_questions", ascending=False),
```

```
        x="theme",
        y="n_questions",
        text="pct_of_general",
        title="Cross-Cutting Themes in General Questions",
        labels={
            "theme": "Theme",
            "n_questions": "Number of Questions"
        }
    )

    fig.update_traces(
        texttemplate="%{text:.2f}%",
        textposition="outside"
    )

    fig.update_layout(
        template="plotly_white",
        yaxis_title="Number of Questions",
        xaxis_title="Theme"
    )

    fig.show()
```




```
con.execute("""
CREATE OR REPLACE TABLE response_level AS
SELECT
  f.question_id,
  l.question_type,
  l.question_topic,
  f.question_language,
  f.response_id,
  f.response_language,
  LOWER(COALESCE(f.response_content, '')) AS response_text
FROM farmer_questions f
JOIN challenge4_labeled l
  ON f.question_id = l.question_id
WHERE f.response_id IS NOT NULL;
""")
```

```
<duckdb.duckdb.DuckDBPyConnection at 0x7b61aa7d2e70>
```

It creates a new table where each row represents a single response, joined to its question, labeled with the question's type and topic, and with the response text cleaned to lowercase so responses can be analyzed directly.

```
con.execute("""
SELECT question_type, COUNT(*) AS n_responses
FROM response_level
GROUP BY 1
ORDER BY n_responses DESC;
""").df()
```

	question_type	n_responses
0	Crop-Specific	8058356
1	Livestock-Specific	6062275
2	General	2641455
3	Unknown	5028

This query counts how many total responses exist for each question type (Crop-Specific, Livestock-Specific, General, Unknown) in the response_level table and orders them from most to least responses.

```
resp_specificity_pct = con.execute("""
WITH labeled AS (
  SELECT
    question_type,
    CASE
      WHEN response_text LIKE '%' || LOWER(question_topic) || '%'
        THEN 'Topic-Matched (More Specific)'
      ELSE 'Not Topic-Matched (More General)'
    END AS response_specificity
  FROM response_level
  WHERE question_topic IS NOT NULL
    AND question_topic <> ''
)
SELECT
  question_type,
  response_specificity,
  COUNT(*) AS n_responses,
  ROUND(
```

```
    100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY question_type),
    2
  ) AS pct_within_question_type
FROM labeled
GROUP BY 1, 2
ORDER BY question_type, pct_within_question_type DESC;
""").df()

resp_specificity_pct
```

	question_type	response_specificity	n_responses	pct_within_question_type
0	Crop-Specific	Not Topic-Matched (More General)	7300527	90.60
1	Crop-Specific	Topic-Matched (More Specific)	757829	9.40
2	General	Not Topic-Matched (More General)	2081707	78.81
3	General	Topic-Matched (More Specific)	559748	21.19
4	Livestock-Specific	Not Topic-Matched (More General)	5720347	94.36
5	Livestock-Specific	Topic-Matched (More Specific)	341928	5.64
6	Unknown	Not Topic-Matched (More General)	4901	97.47
7	Unknown	Topic-Matched (More Specific)	127	2.53

Next steps: Generate code with `resp_specificity_pct` New interactive sheet

This query classifies each response as more specific or more general based on whether the response text explicitly mentions the question's topic, then counts and calculates the percentage of those response types within each question category.

From the output, responses are mostly general across all categories: for crop specific questions, about 7.3 million responses or 90.6 percent do not mention the crop by name, while only 757,816 responses or 9.4 percent are topic matched; livestock specific questions show an even stronger pattern, with 94.36 percent of responses not mentioning the animal and only 5.64 percent being topic matched; general questions have the highest relative specificity, but still only 21.19 percent of responses reference a specific topic, meaning nearly 4 out of 5 general responses remain broad. This directly answers the Challenge 4 question about how farmers are responded to, showing that answers are overwhelmingly generic rather than tailored to the specific crop or animal asked about.

```
con.execute("""
WITH labeled AS (
  SELECT
    question_id,
    question_type,
```

```
    CASE
      WHEN response_text LIKE '%' || LOWER(question_topic) || '%'
        THEN 1 ELSE 0
    END AS is_topic_matched
  FROM response_level
  WHERE question_topic IS NOT NULL
    AND question_topic <> ''
)
SELECT
  question_type,
  COUNT(DISTINCT question_id) AS n_questions_with_responses,
  COUNT(DISTINCT CASE WHEN is_topic_matched=1 THEN question_id END) AS n_questions_with_topic_matched_response,
  ROUND(
    100.0 * COUNT(DISTINCT CASE WHEN is_topic_matched=1 THEN question_id END)
    / COUNT(DISTINCT question_id),
    2
  ) AS pct_questions_with_topic_matched_response
FROM labeled
GROUP BY 1
ORDER BY n_questions_with_responses DESC;
""").df()
```

	question_type	n_questions_with_responses	n_questions_with_topic_matched_response	pct_questions_with_topic_matched_response
0	Crop-Specific	2141561	311455	14.54
1	Livestock-Specific	1604682	176505	11.00
2	General	445954	158386	35.52
3	Unknown	1613	76	4.71

This query measures, by question type, how many unique questions received at least one response that explicitly mentions the question's topic, and what percentage of questions have such a topic-matched response.

From the output, general questions are the most likely to receive topic-matched responses, with 158,387 out of 445,954 questions (35.52 percent) having at least one response that mentions the topic. In contrast, crop specific questions have 311,454 topic-matched questions out of 2,141,563 total (14.54 percent), and livestock specific questions have 176,505 out of 1,604,680 (11.00 percent). This shows that while crop and livestock questions dominate in volume, most of them do not receive responses that directly reference the specific crop or animal, whereas general questions are more than twice as likely to get a topic-matched response.

```
resp_specificity_pct = con.execute("""
WITH labeled AS (
  SELECT
```

```
    question_type,
    CASE
      WHEN response_text LIKE '%' || LOWER(question_topic) || '%'
        THEN 'Topic-Matched (More Specific)'
      ELSE 'Not Topic-Matched (More General)'
    END AS response_specificity
  FROM response_level
  WHERE question_topic IS NOT NULL
    AND TRIM(question_topic) <> ''
)
SELECT
  question_type,
  response_specificity,
  COUNT(*) AS n_responses,
  ROUND(
    100.0 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY question_type),
    2
  ) AS pct_within_question_type
FROM labeled
GROUP BY 1,2
ORDER BY question_type, pct_within_question_type DESC;
""").df()

resp_specificity_pct
```

	question_type	response_specificity	n_responses	pct_within_question_type
0	Crop-Specific	Not Topic-Matched (More General)	7300527	90.60
1	Crop-Specific	Topic-Matched (More Specific)	757829	9.40
2	General	Not Topic-Matched (More General)	2081707	78.81
3	General	Topic-Matched (More Specific)	559748	21.19
4	Livestock-Specific	Not Topic-Matched (More General)	5720347	94.36
5	Livestock-Specific	Topic-Matched (More Specific)	341928	5.64

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.